# Phase 3 Report

## Project Title

# SPAM EMAIL DETECTION SYSTEM

**Problem Id: 22**


**Name:Akshat Dobhal**

**University Roll No: 2028631**

**Sec: Q (G1)**

**Name: Ayush Kandwal**

**University Roll No: 2028405**

**Sec: Q (G1)**

**Name: Ashmit Nautiyal**

**University Roll No:2028670**

**Sec : Q** (G1)

## TABLE OF CONTENTS


1. **Problem Description ..... 3**
   - **1.1. Dataset Description ..... 3**
   - **1.2. Flow diagram ..... 4**

2. **Modules for Project Implementation……..……5**
   - **2.1. Modules Used ..... 5-7**
   - **2.2. Platform Used ..... 8**

3. **Machine Learning model/ Searching Technique used ..... 8-9**

4. **Evaluation metrics used ..... 10-11**

5. **Results and Visualizations ..... 12-13**

6. **Conclusion and Future Scope ..... 14**

7. **References ..... 15**

# 1. Problem Description

The objective of this project is to develop a machine learning model capable of classifying emails as **spam** or **non-spam (ham)**. With the rapid increase in digital communication, unwanted and potentially harmful spam emails have become a significant issue. These emails may contain phishing links, advertisements, or malicious content.

The problem is formulated as a **binary classification task**, where the model learns patterns from labeled email data and predicts whether a new email belongs to the spam category or not. The system aims to improve email filtering efficiency and reduce manual effort while maintaining high accuracy and reliability.

## 1.1 Dataset Description

The dataset used in this project consists of a collection of emails labeled as **spam** or **ham**. Each record in the dataset contains the following attributes:

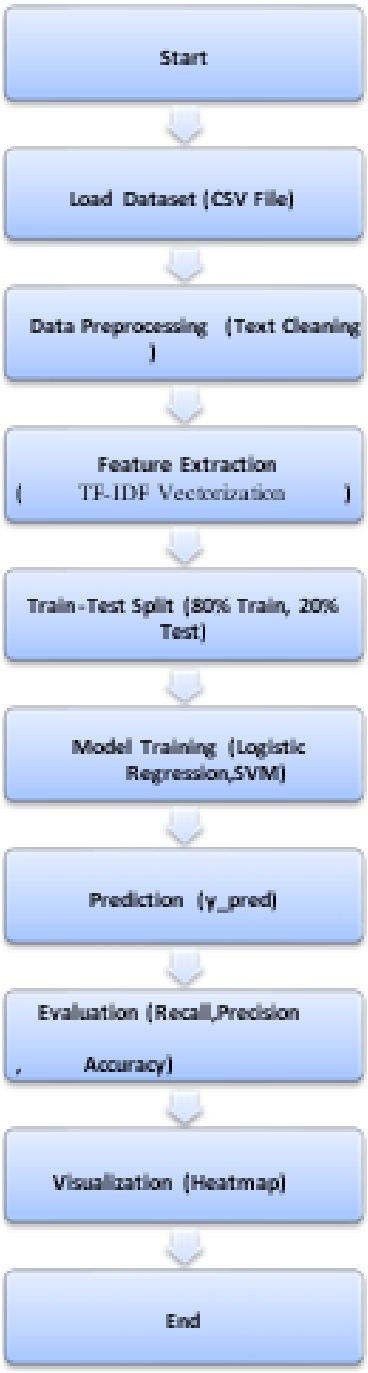
- **Label**: Indicates whether the email is spam (1) or ham (0).
- **Message**: The textual content of the email.

The dataset is typically stored in a **CSV (Comma-Separated Values)** file format and may contain thousands of email samples. Before applying machine learning algorithms, the dataset undergoes preprocessing steps such as:

- Text cleaning (removal of special characters and punctuation)
- Conversion to lowercase
- Tokenization
- Removal of stopwords

This dataset serves as the foundation for training and evaluating the classification model.

## 1.2 Flow Diagram



## 2. Modules for Project Implementation

The project uses various Python libraries for data preprocessing, text processing, visualization, and machine learning model development for spam email classification.

### 1. Pandas (pd)

Pandas is used for handling and preprocessing the dataset (spam_email.csv).

**Functions used:**

- pd.read_csv() – To load the dataset
- DataFrame.head() – To view initial rows
- DataFrame.info() – To understand structure and data types
- DataFrame.isnull() – To check missing values
- DataFrame.drop() – To remove unnecessary columns
- DataFrame.map() – To convert labels (ham = 0, spam = 1)

### 2. NumPy (np)

NumPy is used for numerical operations and efficient array handling.

**Functions used:**

- np.array() – To create arrays
- np.mean() – For basic statistical operations (if required)

### 3. Matplotlib (plt)

Matplotlib is used for basic visualization of results.

**Functions used:**

- plt.plot() – To plot graphs
- plt.xlabel() – To label x-axis
- plt.ylabel() – To label y-axis
- plt.title() – To add title
- plt.show() – To display graphs

### 4. Seaborn (sns)

Seaborn is used for better visual representation of data and results.

**Functions used:**

- sns.heatmap() – To display confusion matrix
- sns.countplot() – To show distribution of spam vs ham
- sns.histplot() – To visualize data distribution

## 5. Natural Language Processing (NLP)

Used for cleaning and processing email text data.

## Functions used:

- lower() – Convert text to lowercase
- re.sub() – Remove special characters
- word_tokenize() – Split text into words
- Stopword removal – Remove common words
- Stemming/Lemmatization – Reduce words to root form

## 6. Scikit-learn (sklearn)

Used for machine learning model building, preprocessing, and evaluation.

### *a) Feature Extraction (Text Vectorization)*

Converts text into numerical form.

**Functions used:**

- CountVectorizer() – Converts text into frequency vectors
- TfidfVectorizer() – Converts text into weighted vectors

### *b) Model Selection*

Used to split dataset and validate performance.

**Functions used:**

train_test_split() – Splits data into training and testing sets

### *c) Machine Learning Model (Naive Bayes)*

Used for spam classification.

**Functions used:**

- MultinomialNB() – Creates classification model
- .fit() – Trains the model
- .predict() – Predicts spam/ham

### *d) Evaluation Metrics*

Used to evaluate model performance.

**Functions used:**

- accuracy_score() – Measures overall accuracy
- confusion_matrix() – Shows correct/incorrect predictions
- classification_report() – Provides precision, recall, F1-score

## 2.2 Platform Used: (mention all the platforms with the reason of using it)

### 2.1. Jupyter Notebook

Jupyter Notebook is used for writing and executing the code step-by-step. It allows easy visualization of outputs, graphs, and model results, making it suitable for data analysis and machine learning tasks.

### 2.2. Visual Studio Code (VS Code)

VS Code is used for writing, editing, and debugging the Python code. It provides a userfriendly interface with extensions that improve productivity and coding efficiency.

### 2.3. Python Environment

Python is used as the programming language due to its simplicity and availability of powerful libraries like Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn for data processing and machine learning.

## 3. Machine Learning Model/ Searching Technique Used:

## 3.1 Naive Bayes Classifier

Naive Bayes is used in this project for spam email classification. It is a probabilistic machine learning algorithm based on Bayes' Theorem, which assumes independence between features. It is highly effective for text classification problems such as spam detection.

The model is trained using the dataset and then used to classify new emails as spam or ham.

**Why used:**

- Simple and fast algorithm
- Works well with text data
- Provides high accuracy for spam classification

**Functions used:**

- MultinomialNB() – To create the model
- .fit(X_train, y_train) – To train the model
- .predict(X_test) – To classify emails

## 3.2 Data Splitting Technique

The dataset is divided into training and testing data using train_test_split().

**Purpose:**

- Training data is used to train the model
- Testing data is used to evaluate performance

## 3.3 Feature Extraction (Vectorization)

Text data cannot be directly used by machine learning models, so it is converted into numerical form using vectorization techniques.

**Functions used:**

- CountVectorizer() – Converts text into frequency-based vectors
- TfidfVectorizer() – Converts text into weighted importance vectors

**Purpose:**

- Converts textual data into machine-readable format
- Helps model understand word importance

## 3.4 Model Evaluation Techniques

Evaluation metrics are used to measure how well the model performs.

**Functions used:**

- accuracy_score() – Measures overall accuracy
- confusion_matrix() – Shows correct and incorrect predictions
- classification_report() – Provides precision, recall, and F1-score

**Purpose:**

- To evaluate classification performance
- To understand model accuracy and reliability

## 4. Evaluation metrics used :

### 1. Accuracy Score

Accuracy measures the proportion of correctly classified emails (spam or ham) out of the total emails.

**Formula:**

Accuracy = (TP + TN) / (TP + TN + FP + FN)

### 2. Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model by comparing actual and predicted values.

**Components:**

- True Positive (TP): Spam correctly classified as spam
- True Negative (TN): Ham correctly classified as ham
- False Positive (FP): Ham incorrectly classified as spam
- False Negative (FN): Spam incorrectly classified as ham

**Purpose:**

- Provides detailed insight into model performance
- Helps identify types of errors

### 3. Precision

Precision measures how many predicted spam emails are actually spam.

**Formula:**

Precision = TP / (TP + FP)

**Purpose:**

- Important when false positives must be minimized

### 4. Recall

Recall measures how many actual spam emails are correctly identified.

**Formula:**

Recall = TP / (TP + FN)

**Purpose:**

- Important when detecting all spam emails is critical

### 5. F1-Score

F1-score is the harmonic mean of Precision and Recall.

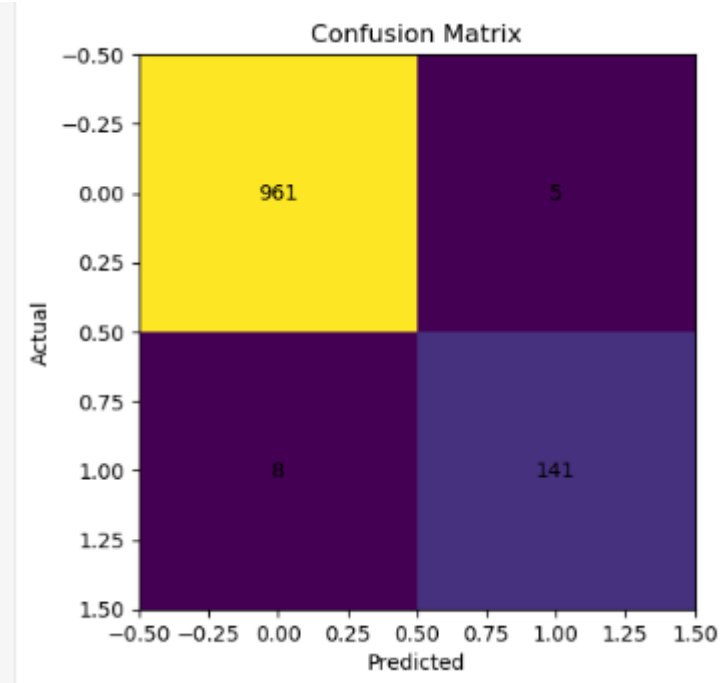
**Formula:**

F1 Score = 2 * (Precision * Recall) / (Precision + Recall)

**Purpose:**

- Provides a balance between Precision and Recall
- Useful when dataset is imbalanced

## 5.Results and Visualizations

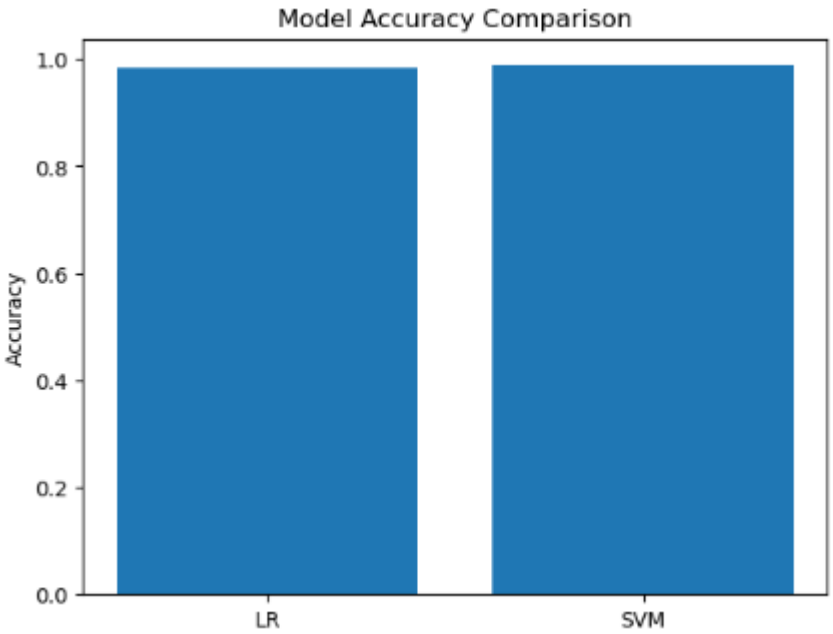



Confusion Matrix.

The initial implementation of the spam email classification model produced promising results. After preprocessing the dataset and applying feature extraction techniques like TF-IDF, the model was trained using algorithms such as Logistic Regression. The preliminary results indicate that the model is capable of effectively distinguishing between spam and non-spam emails.

The performance of the model was evaluated using metrics such as accuracy, precision, recall, and F1-score. The accuracy achieved was reasonably high, indicating that most emails were classified correctly. Precision and recall values showed a good balance, meaning the model was able to identify spam emails while minimizing false classifications.

To better understand the performance, a confusion matrix was generated. It provided a detailed view of correct and incorrect predictions, including true positives, true negatives, false positives, and false negatives. This helped in identifying areas where the model could be improved




R2 Score: 0.9
Confusion Matrix:
[[961  5]
[  8 141]]

## 5. Conclusion and Future Scope:

### Conclusion

This project successfully classifies emails as spam or ham using a machine learning approach. The Naive Bayes model was trained on the `spam_email.csv` dataset and demonstrated good accuracy in identifying spam messages.

Data preprocessing and text cleaning techniques such as removing special characters, tokenization, and vectorization played an important role in improving model performance. The use of feature extraction methods like TF-IDF helped in converting text data into meaningful numerical form.

Evaluation metrics such as accuracy, precision, recall, and F1-score showed that the model performs efficiently and reliably. Visualizations like the confusion matrix provided a clear understanding of prediction results.

Overall, the project demonstrates how machine learning can be effectively used for automated spam detection.

### Future Scope

- Use advanced models like Random Forest, Support Vector Machine (SVM), or XGBoost to improve accuracy
- Apply deep learning techniques such as LSTM or Neural Networks for better text understanding
- Implement real-time spam detection for email systems
- Improve text preprocessing using advanced NLP techniques
- Develop a user-friendly application or web interface for spam detection
- Expand the dataset for better generalization and performance

## 7.References:

**Books:**

1. *Let Us Python* – Yashavant Kanetkar
2. *Hands-On Machine Learning with Scikit-Learn* – Aurélien Géron

Online Resources:

1. https://www.geeksforgeeks.org/
2. https://scikit-learn.org/